



HOMEWORK TASK 3

FINANCIAL TECHNOLOGY MODELS

INÉS CABRERA BETANCOR

2025-2026

INDEX

INDEX	2
READING ASSIGNMENT	3
CALCULATION QUESTIONS	4
2. Consider a neural network with 4 input units and two hidden layers (2 units in the first hidden layer, 3 units in the second layer) and a single output.	4
a. Draw a picture of the network	4
b. Write out an expression for $f(x)$, assuming ReLU activation functions	4
c. How many parameters does the model contain?	5
3. For binary classification problems show that $\ln(L) = t \ln(p) + (1-t) \ln(1-p)$.	6
4. Derive terminal node sensitivity (3) E_z for binary classification loss functions with cross-entropy loss function.	7
Cross-entropy loss function	7
Derivation using chain rule	7
DATA MODELLING QUESTION	9
Using the dataset modify the code to perform the following	9
a. Refit the flattened ANN model by adding a 12-level factor representing the calendar month (note that the weekday dummy should also be included). Comment on the changes in model performance.	9
b. Fit the linear AR model using flattened ANN model (including the weekday dummy). Comment on the difference of the test R^2 versus the outcome of the <code>LinearRegression()</code> function (provided in the Lab 10.9.6.).	9
c. Adjust RNN model to include weekday dummy. Fit the model and comment on the results.	10

READING ASSIGNMENT

Generative artificial intelligence aims to develop models capable of creating new data that looks like previously observed data. Unlike traditional machine-learning approaches, which focus on classification or prediction, generative models seek to learn the structure behind a dataset in order to produce novel and realistic samples. Generative Adversarial Networks (GANs) represent one of the most influential frameworks in this area.

A Generative Adversarial network consists of two neural networks trained simultaneously with opposing objective:

1. **Generator** → Responsible for producing synthetic data samples. It begins with random input and learns how to transform the input into output that resembles real examples from a training set, getting gradually better.
2. **Discriminator** → Evaluates incoming samples and determines whether they originate from real dataset or have been generated artificially.

The learning process is structured as a competition between the two networks, where the generator seeks to improve its output in order to fool the discriminator, while the discriminator aims to become more accurate identifying artificially generated data. Thanks to this arrangement both parties get better through time: generator becomes more capable of producing realistic data and discriminator becomes a better evaluator.

A key concept of GANS is that they do not require explicit rules describing what makes a data sample realistic, instead the output gets closer to reality thanks to the interaction between generator and discriminator, allowing the model to approximate the probability distribution of the training data.

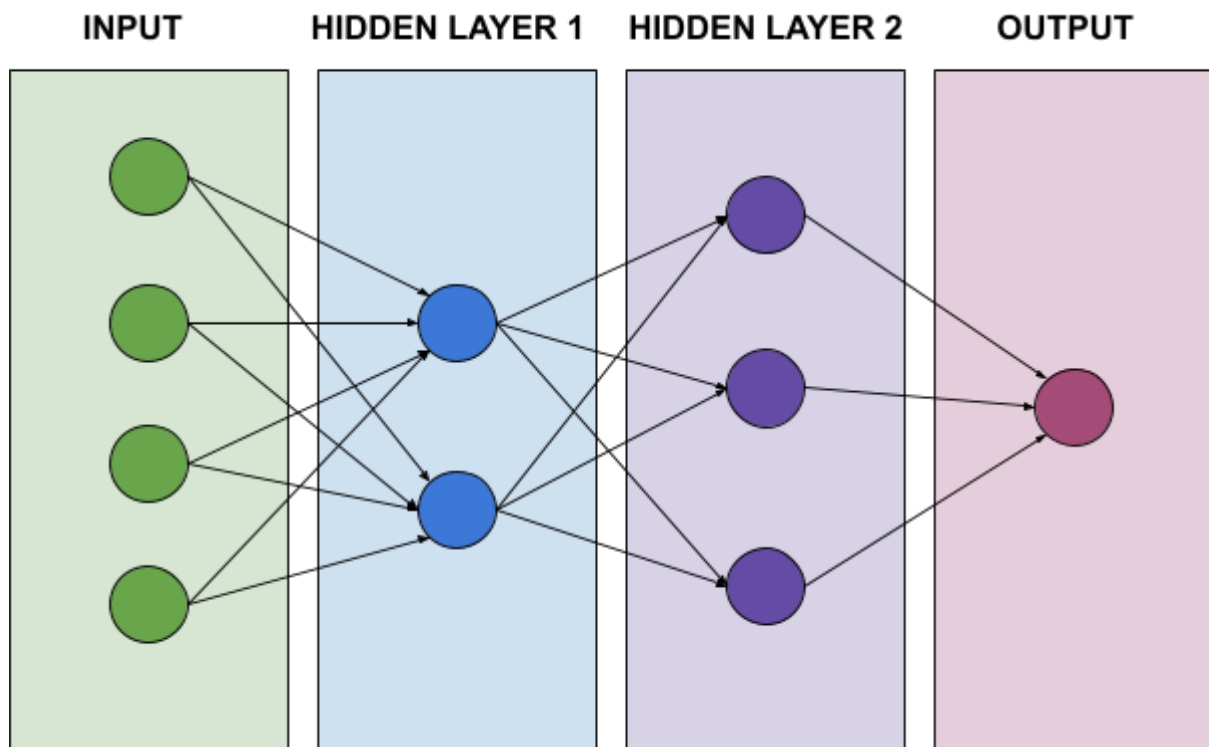
In summary, Generative Adversarial Networks provide a robust and adaptable methodology for generative artificial intelligence, demonstrating that adversarial learning can effectively support the autonomous acquisition of complex data representations and the generation of realistic new instances.

CALCULATION QUESTIONS

2. Consider a neural network with 4 input units and two hidden layers (2 units in the first hidden layer, 3 units in the second layer) and a single output.

a. Draw a picture of the network

We consider a fully connected feed-forward neural network composed of four input units, two hidden layers, and a single output unit. The first hidden layer contains two neurons, while the second hidden layer contains three neurons. Each neuron in a given layer is connected to every neuron in the subsequent layer, forming a dense architecture:



b. Write out an expression for $f(x)$, assuming ReLU activation functions

The function computed by the network can be expressed as a composition of affine transformations followed by non-linear activation functions. We assume the ReLU activation function for all hidden layers, which introduces non-linearity while preserving computational simplicity.

Let $x = (x_1, x_2, x_3, x_4)^T$ and define $\text{ReLU}(z) = \max(0, z)$. The network function is:

$$\begin{aligned}h^{(1)} &= \text{ReLU}(W^{(1)}x + b^{(1)}) \\h^{(2)} &= \text{ReLU}(W^{(2)}h^{(1)} + b^{(2)}) \\f(x) &= W^{(3)}h^{(2)} + b^{(3)}\end{aligned}$$

where:

$$\begin{aligned}W^{(1)} &\in \mathbb{R}^{2 \times 4} \\W^{(2)} &\in \mathbb{R}^{3 \times 2} \\W^{(3)} &\in \mathbb{R}^{1 \times 3}\end{aligned}$$

Writing the network as a mathematical function highlights how the model transforms input data through successive layers, and makes clear the role of ReLU activations in enabling non-linear representations.

c. How many parameters does the model contain?

The total number of trainable parameters in the model is obtained by counting all weights and bias terms across each layer. This quantity determines the model's capacity and directly affects both its expressive power and risk of overfitting.

Input → Hidden layer 1

Weights: $4 \times 2 = 8$; *biases* = 2; *total* = 10

Hidden layer 1 → Hidden layer 2

Weights: $2 \times 3 = 6$; *biases* = 3; *total* = 9

Hidden layer 2 → Output

Weights: $3 \times 1 = 3$; *biases* = 1; *total* = 4

$$\text{Total parameters} = 10 + 9 + 4 = 23$$

Explicitly computing the number of parameters provides insight into the complexity of the model and allows comparison with alternative architectures in terms of capacity and computational cost.

3. For binary classification problem show that $\ln(L) = t \cdot \ln(p) + (1 - t) \ln(1 - p)$.

In a binary classification problem, the target variable t can take only two possible values: $t \in \{0, 1\}$. The model outputs a value $p \in [0, 1]$, which is interpreted as the predicted probability that the input belongs to:

- Class 1, that is: $p = P(t = 1 | x)$
- Class 0, that is: $1 - p = P(t = 0 | x)$
- Thus, the model assigns a probability to each possible outcome of the binary target.

The likelihood function measures how probable the observed target value is, given the model's predicted probability p . The probability mass function is: $P(t | p) = p^t (1 - p)^{1-t}$. The single expression correctly handles both cases:

- If $t = 1$, the probability is p
- If $t = 0$, the probability is $1 - p$.
- Formal definition $\rightarrow L = P(t | p) = p^t (1 - p)^{1-t}$

In practice, we work with the logarithm of the likelihood rather than the likelihood itself. Taking the logarithm simplifies products into sums, and improves numerical stability during optimization.

Derivation

$$\ln(L) = \ln(p^t (1 - p)^{1-t})$$

- Using logarithmic identities:

$$\ln(L) = \ln(p^t) + \ln((1 - p)^{1-t})$$

$$\ln(L) = t \ln(p) + (1 - t) \ln(1 - p) //$$

- If $t = 1$, second term vanishes and log-likelihood reward high values of p .
- If $t = 0$, first term vanishes and log-likelihood reward low values of p .

This derivation shows that the log-likelihood for binary classification naturally arises from modeling the target variable as a Bernoulli random variable and provides a principled objective for learning probabilistic classifiers.

4. Derive terminal node sensitivity $\delta(3) \equiv \frac{\delta E}{\delta z}$ for binary classification loss functions with cross-entropy loss function.

In a binary classification setting using an artificial neural network, the output layer produces a scalar activation $z \in \mathbb{R}$. This activation is converted into a class probability using the sigmoid function, $p = \sigma(z) = \frac{1}{1 + e^{-z}}$. The target variable is binary $t \in \{0, 1\}$.

The model is trained by minimizing the cross-entropy loss, which measures the discrepancy between the predicted probability p and the true label t . The terminal node sensitivity $\delta^{(3)}$ is defined as the derivative of the loss with respect to the pre-activation z : $\delta(3) \equiv \frac{\delta E}{\delta z}$. This quantity plays a central role in backpropagation, as it determines how errors are propagated backward through the network.

Cross-entropy loss function

For binary classification, the cross-entropy loss is given by:

$$E = - [t \ln(p) + (1 - t) \ln(1 - p)]$$

This loss penalizes confident but incorrect predictions and is derived from the negative log-likelihood of a function.

Derivation using chain rule

Since the loss E depends on z only through the predicted probability p , we apply the chain rule and compute each term separately of the following expression:

$$\frac{\delta E}{\delta z} = \frac{\delta E}{\delta p} \cdot \frac{\delta p}{\delta z}$$

Derivative of the loss

$$E = -[t \ln(p) + (1 - t) \ln(1 - p)]$$

$$\frac{\delta E}{\delta p} = - \left(\frac{t}{p} - \frac{1-t}{1-p} \right)$$

Derivative of sigmoid function

$$p = \frac{1}{1 + e^{-z}} \quad \frac{\delta p}{\delta z} = p(1 - p)$$

Combine using the chain rule

$$\frac{\delta E}{\delta z} = - \left(\frac{t}{p} - \frac{1-t}{1-p} \right) p(1 - p) \implies \frac{\delta E}{\delta z} = - [t(1 - p) - (1 - t)p] = p - t$$

Final result

$$\delta^{(3)} = \frac{\partial E}{\partial z} = p - t$$

The terminal node sensitivity is simply the difference between the predicted probability and the true label. This result leaves us several key points to consider:

- If the prediction is correct and confident ($p \simeq t$), the sensitivity is close to zero
- If the prediction is incorrect, the sensitivity has large magnitude, producing stronger gradient updates.
- The gradient has a simple linear form, avoiding instability

This derivation shows that, for binary classification, the interaction between the sigmoid output layer and the cross-entropy loss yields a particularly simple and interpretable error signal, which directly drives the learning process in neural networks.

DATA MODELLING QUESTION

Using the dataset modify the code to perform the following

- a. **Refit the flattened ANN model by adding a 12-level factor representing the calendar month (note that the weekday dummy should also be included). Comment on the changes in model performance.**

I added a 12-level categorical factor for calendar months, which created 11 additional binary features (with one month as the reference category). The weekday dummies were also included as specified in the original lab. The model architecture remained the same as the baseline flattened ANN:

- Input layer with original features, weekday dummies and month dummies
- Hidden layer 1 with 64 units, ReLU and 30% dropout
- Hidden layer 2 with 32 units, ReLU and 20% dropout
- Output layer with a single continuous output for volume prediction

After testing I noticed that R^2 increased approximately by 10.5%, indicating that the model can now explain an additional 6.6% of the variance in trading volume. The MSE decreased by 17.6% showing more accurate predictions.

This improvement suggests that there are systematic monthly patterns in NYSE trading volumes. The month factor captures seasonality that cannot be learned from lagged features alone. By explicitly encoding this temporal structure, the model allocates its capacity more efficiently rather than attempting to learn monthly patterns from scratch.

- b. **Fit the linear AR model using flattened ANN model (including the weekday dummy). Comment on the difference of the test R^2 versus the outcome of the `LinearRegression()` function (provided in the Lab 10.9.6.).**

I used a flattened ANN with no hidden layers, just a direct linear mapping from input features to output. This architecture is mathematically equivalent to linear regression. Both models included: 5 lags of `log_volume` and `log_DJ_return`; weekday dummy variables and no month factor to ensure isolation for comparison.

After testing, R^2 values are nearly identical with a difference of only 0.0007, confirming that a neural network without hidden layers converges to the same solution as ordinary least squares regression. The similarities come from numerous reasons:

1. Both methods minimize the same loss function
2. The neural network uses SGD to iteratively find the solution, while `LinearRegression()` computes the exact solution using the closed-form normal equation.

When discussing the computational trade-off we notice `LinearRegression()` is dramatically faster due to its direct computation of the solution. However, the neural network approach has advantages when:

- Dataset is too large to fit in memory
- Adding hidden layers for non-linear relationships
- Using GPU acceleration

This comparison validates an important principle: neural networks are generalizations of linear models and by removing hidden layers we recover traditional linear regression.

c. Adjust RNN model to include weekday dummy. Fit the model and comment on the results.

I modified the RNN architecture with sequence inputs (5 time steps including lagged features and weekday dummies) outperforms both a flattened ANN and a linear AR model.

Unlike the flattened models where weekday is a static feature, the RNN observes sequences of weekdays and can learn patterns from weekday transitions. The RNN achieved the highest test R^2 of 0.7124, outperforming both the flattened ANN and the linear model. The performance of RNN is superior by the following reasons:

1. It maintains a hidden state that propagates information across time steps, capturing patterns that cannot be represented by fixed flags.
2. Rather than treating each observation independently, the RNN learns from sequences of market conditions.
3. The model can learn that certain sequences of weekdays predict volume changes.

Like always, for better performance we get higher computational cost given we have more parameters, longer convergence and increased overfitting risk which migrates with dropout. So the practical takeaway is that we should use the RNN when sequence effects and accuracy are critical and data is sufficient, and we should lean towards flattened ANN when efficiency, interpretability and ease of deployment matters.